



Практическое задание

Тема: Мини-REST API “Заметки” + клиент на `HttpClient`

Цель

- Потренировать модель **запрос–ответ**
- Закрепить **методы (GET/POST/DELETE)**, коды статуса, заголовки, **JSON-тело**
- Научиться проверять `StatusCode` и (де)сериализовать данные

Задание (шаги)

1) Сервер (**ASP.NET Core Minimal API**)

Создайте ресурс **Note**:

- `Id:int`
- `Text:string`

Сделайте 3 endpoint'a:

1. `GET /notes`

Возвращает список заметок.

Ответ: `200 OK` + JSON-массив.

2. `POST /notes`

Принимает JSON: `{ "text": "..."}` и создаёт заметку.

Проверка: `text` не пустой.

Ответы:

- `201 Created` + JSON созданной заметки (и желательно `Location: /notes/{id}`)
- `400 Bad Request` + `{ "error": "Text is required" }`

3. `DELETE /notes/{id}`

Удаляет заметку по `id`.

Ответы:

- `204 No Content` если удалили
- `404 Not Found` если такой нет

Хранение данных: **просто в памяти** (например, `List<Note>` + счётчик `id`).

2) Клиент (консольное приложение)

Сценарий в коде:

1. `POST /notes` — создать 2 заметки
2. `GET /notes` — вывести список в консоль
3. `DELETE /notes/{id}` — удалить первую
4. `GET /notes` — вывести список ещё раз

Подсказки по ключевым частям

- **POST** должен отправлять **JSON** и заголовок `Content-Type: application/json`
- **Клиент всегда проверяет** `response.StatusCode` **до** чтения/десериализации тела
- **DELETE** обычно без тела, успешное удаление — `204`

Что проверить перед отправкой (чек-лист)

- ☐ `GET /notes` возвращает `200` и JSON
- ☐ `POST /notes` возвращает `201`, а при пустом `text` — `400` с JSON-ошибкой
- ☐ `DELETE /notes/{id}` возвращает `204`, а при несуществующем `id` — `404`
- ☐ Клиент не падает, а корректно реагирует на `400/404`
- ☐ В клиенте выводятся заметки до и после удаления

Советы по улучшению работы (по желанию)

- Добавьте `GET /notes/{id}` (`200` или `404`)
- Добавьте заголовок `X-Request-Id` (GUID) в ответы сервера
- В клиенте вынесите запросы в класс `NotesApiClient`